

Virtual Memory Replacement & Allocation: Efficiency–Fairness Trade-offs Under Memory Pressure

Pranav Kumar Kaliaperumal, Akanksha Gutal, Spandana Erukulla
Department of Computer Science and Engineering
CSCI 5573: Operating Systems
University of Colorado Denver

Abstract—This report presents a custom virtual memory simulator and trace-driven benchmarking framework for evaluating page replacement and frame allocation policies under multiprogrammed memory pressure. We implemented FIFO, LRU, NRU, Second Chance, and WSClock replacement together with global, local, and proportional allocation. Using synthetic and trace-driven workloads, repeated trials, and fairness-oriented runtime analysis, we evaluated policy behavior across multiple memory-pressure levels. The final campaign shows that H1 and H2 are supported, indicating that policies that improve fault behavior under pressure can still degrade fairness near the working-set boundary, while H3 is not supported. These results show that page-fault minimization alone is not sufficient to identify the best policy under mixed, fairness-sensitive workloads.

I. INTRODUCTION

Virtual memory is one of the core abstractions in operating systems: it separates the address space seen by a process from the limited physical frames available in the machine. Demand paging lets programs exceed physical memory capacity, but it also forces the operating system to decide which resident pages to evict and how many frames each process should receive. These decisions directly affect page-fault frequency, disk activity, runtime, and the quality of service received by competing processes.

This project connects directly to the operating-systems course topics of paging, page replacement, working sets, frame allocation, multiprogramming, and systems measurement. The course treatment explains the mechanisms; this report evaluates their interactions experimentally. In particular, it asks whether a policy that improves global efficiency can also hurt fairness when multiple processes compete near their aggregate working-set size (WSS).

Prior work established the theoretical importance of replacement choices and locality, including Belady’s analysis of replacement algorithms and Denning’s working-set model [1], [2], [3]. Modern systems continue to revisit virtual memory overheads, allocation behavior, and measurement sensitivity as machines grow in memory capacity and complexity [4], [5], [6], [7]. Our contribution is a controlled teaching-scale framework that combines replacement policy, allocation policy, workload generation, trace replay, and fairness metrics in one reproducible evaluation.

We freeze the paper around the following claim: we built a custom virtual memory simulation and trace-driven benchmarking framework to evaluate how page replacement and frame allocation policies trade off efficiency and fairness under memory pressure, and we found that H1 and H2 are supported while H3 is not.

The research questions were:

- **RQ1:** Which replacement/allocation pairings reduce page faults without creating unfair slowdown across processes?
- **RQ2:** Does memory pressure near aggregate WSS create a threshold where global allocation stops helping fairness?
- **RQ3:** Can simpler local policies such as FIFO or NRU outperform history-based global policies on fairness for heterogeneous workloads?

The main contributions are:

- a Python virtual memory simulator with five replacement policies and three allocation policies;
- a synthetic and trace-driven experiment harness with reproducible raw, processed, table, figure, and demo artifacts;
- a fairness analysis based on shared-versus-isolated slowdown, worst-case slowdown, variance, coefficient of variation, and Jain’s index;
- a final hypothesis evaluation showing H1 supported, H2 supported, and H3 not supported.

II. RELATED WORK

Belady’s study of virtual-storage replacement algorithms showed that simple eviction policies can behave pathologically, including FIFO cases where adding frames can increase faults [1]. Denning’s working-set model then connected paging behavior to locality by arguing that a process has a time-varying set of pages that must remain resident to avoid excessive faults [2]. Denning’s later survey placed these ideas in the broader design space of virtual memory mechanisms and adaptive policy choices [3].

Frame allocation is the second half of the paging decision. Operating systems texts distinguish local replacement, where each process competes mainly with its own resident set, from

global replacement, where processes compete for a shared pool of frames [8]. Proportional allocation adds a compromise by dividing frames according to process demand. Cache replacement with explicit memory allocation has also been studied as a joint optimization problem rather than as replacement alone [9].

Modern systems keep virtual memory relevant because page tables, memory allocators, and compaction layers all create measurable overheads at scale. Basu et al. studied efficient virtual memory for large-memory servers, Park et al. examined page-table caching pressure, Schwalb et al. addressed persistent-memory allocation, and Powers et al. explored memory compaction for C/C++ programs [4], [5], [10], [6]. These works motivate why the replacement/allocation boundary remains important beyond textbook examples.

This report differs from prior work by focusing on a course-scale but end-to-end empirical framework. It does not propose a new kernel policy. Instead, it contributes a transparent simulator and artifact bundle that make the efficiency-versus-fairness trade-off visible across replacement policy, allocation policy, memory pressure, synthetic workload family, and trace-driven replay.

III. SYSTEM DESIGN AND IMPLEMENTATION

The project is implemented as a custom Python-based simulator rather than as a production-kernel modification. The simulator models page accesses, page tables, physical frames, replacement decisions, allocation boundaries, and per-process accounting. Its design has four main components.

The **workload generator** constructs synthetic memory-reference streams for high-locality, streaming, mixed, and thrashing workloads. These families stress different access behaviors: compact reuse, sequential scans, heterogeneous mixes, and pressure-heavy fault patterns. The same interface also accepts normalized trace events loaded from CSV files under `workloads/traces/`.

The **memory manager** maintains per-process page tables and a configurable physical-frame pool. It supports `GlobalAllocation`, `LocalAllocation`, and `ProportionalAllocation`. Global allocation lets processes compete for a shared frame pool; local allocation isolates each process more strongly; proportional allocation divides memory according to estimated demand.

The **policy engine** implements FIFO, LRU, NRU, Second Chance, and WSClock. These policies cover a range from simple insertion-order eviction to reference-bit approximations and working-set-inspired clock behavior. Each policy exposes a common interface so the experiment runner can vary replacement and allocation independently.

The **experiment and analysis layer** automates repeated runs, exports JSON/CSV summaries, computes slowdown from isolated and shared runtimes, validates processed outputs, and generates report tables and figures. The primary entry points are `main.py`, `experiments/run_all_experiments.py`, `scripts/aggregate_final_results.py`, `scripts/validate_results.py`,

TABLE I
FINAL CAMPAIGN SCOPE.

Factor	Final setting
Replacement algorithms	FIFO, LRU, NRU, Second Chance, WSClock
Allocation policies	GlobalAllocation, LocalAllocation, ProportionalAllocation
Synthetic workloads	high_locality, streaming, mixed, thrashing
Memory levels	0.8x, 1.0x, 1.2x, 1.5x aggregate WSS
Synthetic process counts	2, 4, 8
Repetitions	10 per synthetic configuration
Report-grade aggregate	1005 configuration groups / 10,050 trial rows
Trace families	Five report traces; trace process counts are preserved from the trace files

`scripts/build_report_tables.py`, and `scripts/build_report_figures.py`. Windows and Unix wrapper scripts provide quiet-run and pinned-core execution paths for more stable timing metadata.

Our original proposal described an emulator-based experimental plan; however, during implementation we refined the methodology and built a custom Python-based virtual memory simulator and trace-driven benchmarking framework instead. This gave us tighter control over replacement policies, allocation strategies, workload generation, and measurement instrumentation while preserving the original research goals.

IV. EXPERIMENTAL METHODOLOGY

The final campaign used the locked raw campaign `results/raw/campaign_20260404_140330_266`. Table I summarizes the report-grade scope. The synthetic matrix used five replacement algorithms, three allocation policies, four workload families, four memory-pressure levels, three process counts, and ten repetitions. The trace-driven side used five report traces plus one demo-only trace excluded from report-grade aggregation; trace rows preserve the process counts encoded in the trace rather than forcing the synthetic 2/4/8 grid.

The memory-pressure settings were defined relative to aggregate WSS. This makes the experiments comparable across workloads with different locality footprints. At 1.5x WSS, the system has more slack; at 1.2x and 1.0x, it approaches the working-set boundary; at 0.8x, it operates under strong pressure.

Experiments were run on a Windows host with a 12th Gen Intel Core i5-12400F processor, 16 GB of RAM, and an NVIDIA RTX 3050 GPU. Runtime-sensitive experiments used wrapper scripts such as `scripts/run_quiet_benchmark.ps1` and `scripts/run_pinned.ps1` to capture environment metadata and reduce host interference. These controls improve repeatability, but they do not eliminate all wall-clock noise.

V. METRICS AND STATISTICAL ANALYSIS

The simulator records page faults, page replacements, disk reads, disk writes, total disk activity, and runtime-derived

slowdown. Page faults and disk events represent efficiency. Slowdown is measured by running each workload both in isolation and in a shared multiprogrammed setting, then computing

$$\text{slowdown} = \frac{t_{\text{shared}}}{t_{\text{isolated}}}.$$

Fairness is summarized with worst-case slowdown, slowdown variance, coefficient of variation (CV), and Jain’s fairness index. Jain’s index is close to 1.0 when normalized performance is evenly distributed and lower when one or more processes receive worse service.

Repeated trials support means, standard deviations, confidence-interval style summaries, and threshold-based hypothesis checks. The hypothesis table is generated from `results/processed/hypothesis_evaluation.csv`, while the report-facing policy tables are generated from `results/processed/final_aggregate_results.csv`. The validation bundle `results/processed/validation_report.json` records 1005 valid source groups and no duplicate source or configuration records, but it also records non-blocking instrumentation issues such as missing shared or isolation runtime records for some raw summaries. Therefore, page-fault and fairness-pattern trends are more stable than exact wall-clock slowdown point estimates.

VI. RESULTS AND ANALYSIS

A. Campaign Coverage

The final processed outputs are anchored by `results/processed/final_aggregate_results.csv`, `results/processed/hypothesis_evaluation.csv`, and `results/processed/validation_report.json`. The report-facing tables in `results/tables/` and figures in `results/figures/` were generated from these processed outputs. The campaign covers 720 synthetic aggregate configurations with 7,200 trials and 285 trace aggregate configurations with 2,850 trials. This lets us compare controlled stress patterns against concrete reference streams.

B. Synthetic Workload Results

Table II reports the table-selected best policy combination for each synthetic workload under three metrics; ties are possible in the underlying aggregate. FIFO with global allocation produced the lowest page-fault totals across the synthetic workload families in this aggregate. The fairness-oriented winners were different: NRU with proportional or local allocation often minimized worst-case slowdown, and local or proportional allocation most often produced the highest Jain’s index. This split is the main empirical signal behind the paper: one policy pairing may reduce faults while another better protects the slowest process.

Figure 1 shows the expected pressure effect: tighter memory budgets increase fault sensitivity. The exact policy ranking varies by workload, but the high-level pattern is stable enough

TABLE II
HEADLINE RESULTS BY SYNTHETIC WORKLOAD.

Workload	Lowest page faults	Lowest worst slowdown	Highest Jain’s index
high_locality	FIFO + GlobalAllocation (144.0)	NRU + ProportionalAllocation (1.5105)	SecondChance + LocalAllocation (0.99997)
mixed	FIFO + GlobalAllocation (202.7)	NRU + LocalAllocation (1.5054)	FIFO + LocalAllocation (0.99988)
streaming	FIFO + GlobalAllocation (340.0)	NRU + LocalAllocation (1.4488)	LRU + ProportionalAllocation (0.99991)
thrashing	FIFO + GlobalAllocation (204.0)	NRU + ProportionalAllocation (1.5057)	FIFO + LocalAllocation (0.99987)

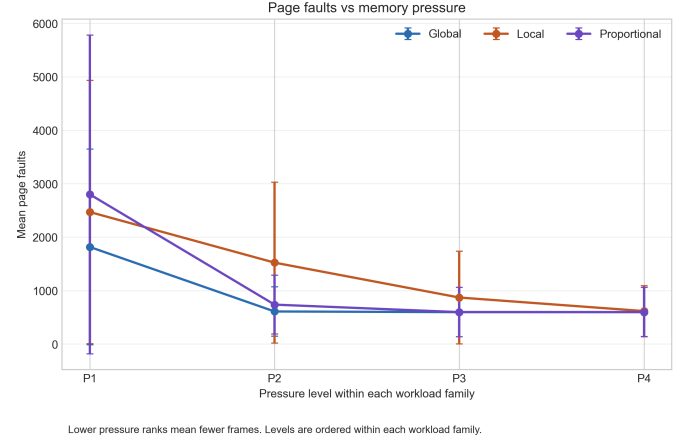


Fig. 1. Page faults versus memory pressure in the final aggregate.

to support the conclusion that fault minimization is not the same as fairness optimization.

C. Trace-Driven Results

Table III summarizes the trace-driven replay results. The trace rows are much smaller than the synthetic matrix, but they are valuable because they verify that the same tooling can replay concrete event streams. FIFO with global allocation again produced the table-selected lowest page-fault counts in the trace rows, while FIFO with proportional allocation minimized worst-case slowdown across the five trace rows. Jain’s-index winners still varied by trace, reinforcing the need to report more than one metric.

D. Efficiency vs. Fairness Trade-offs

Figures 3 and 4 show that allocation policy changes the fairness story. Worst-case slowdown exposes tail behavior that average faults can hide, while Jain’s index captures how evenly service is distributed. A policy that wins on faults can still create a worse slowest process, especially near the working-set boundary.

E. Runtime and Stability Observations

The slowdown metrics are useful for directional fairness comparisons, but they are noisier than page faults. The isolation baseline study in `docs/isolation_stability.md` reports 264 baseline groups, with 81 stable groups, 39 review groups, and 144 unstable groups. The median CV was 0.128607 and the mean CV was 0.1902, so the report treats exact slowdown magnitudes cautiously. This caveat does not invalidate the page-fault trends, and it is why the discussion emphasizes broad fairness patterns and hypothesis directions rather than single-run wall-clock values.

TABLE III
HEADLINE RESULTS BY TRACE WORKLOAD.

Trace workload	Lowest page faults	Lowest worst slowdown	Highest Jain's index
report_locality_heavy	FIFO + GlobalAllocation (4.0)	FIFO + ProportionalAllocation (1.0782)	NRU + GlobalAllocation (0.99967)
report_mixed_heterogeneous	FIFO + GlobalAllocation (11.0)	FIFO + ProportionalAllocation (0.9630)	SecondChance + GlobalAllocation (0.99846)
report_streaming_sequential	FIFO + GlobalAllocation (16.0)	FIFO + ProportionalAllocation (1.0486)	SecondChance + GlobalAllocation (0.99965)
report_thrashing_pressure	FIFO + GlobalAllocation (10.0)	FIFO + ProportionalAllocation (1.0739)	WSClock + LocalAllocation (0.99963)
sample_trace	FIFO + GlobalAllocation (5.0)	FIFO + ProportionalAllocation (0.9342)	LRU + LocalAllocation (0.99781)

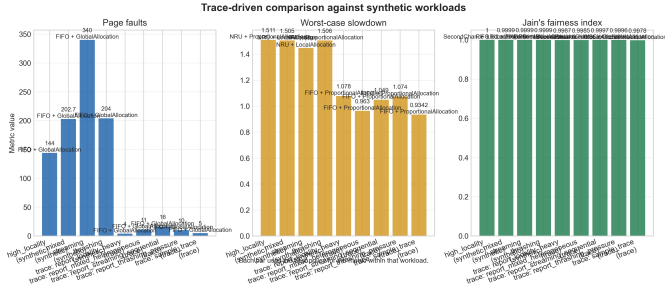


Fig. 2. Trace-driven comparison across report-grade trace families.

VII. HYPOTHESIS OUTCOME DISCUSSION

H1 is supported. The H1 comparison used high-locality workloads with four processes near 1.2x aggregate WSS. LRU with global allocation reduced page faults from 326.2 to 325.1 compared with LRU local allocation, a 0.34% reduction, while worst-case slowdown increased from 1.6298 to 3.6517, a 124.05% increase. Jain's index also dropped from 0.99645 to 0.95621 in that slice. The fault improvement is small, but the slowdown penalty is large, confirming the expected efficiency–fairness trade-off.

H2 is supported. Near the aggregate WSS boundary with four processes, slowdown variance increased from 0.01190 under local allocation to 0.01886 under global allocation, a 58.53% increase. This supports the hypothesis that global allocation can amplify fairness differences near the working-set boundary.

H3 is not supported. For the mixed workload with four processes near aggregate WSS, the FIFO/NRU local comparison condition increased worst-case slowdown from 2.6449 to 3.2209, a 21.78% increase, instead of reducing it. The advanced global comparison also had fewer faults in that slice, moving from 1115.7 to 345.5 faults when compared against the simple local condition. The primary fairness metric did not improve, so the result rejects the claim that simple local policies were better for fairness under the mixed-workload conditions tested.

VIII. LIMITATIONS AND THREATS TO VALIDITY

The first threat is wall-clock timing noise. Shared and isolated runtimes are measured on a general-purpose host, so operating-system scheduling and background activity can affect slowdown values even with quiet and pinned wrappers.

The second threat is trace diversity. The trace-driven study covers five report traces, which is enough to exercise replay support but not enough to represent the full range of real production workloads.

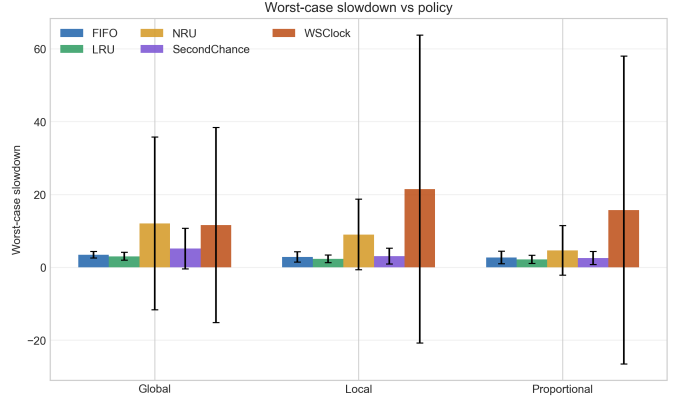


Fig. 3. Worst-case slowdown by allocation policy in the final aggregate.

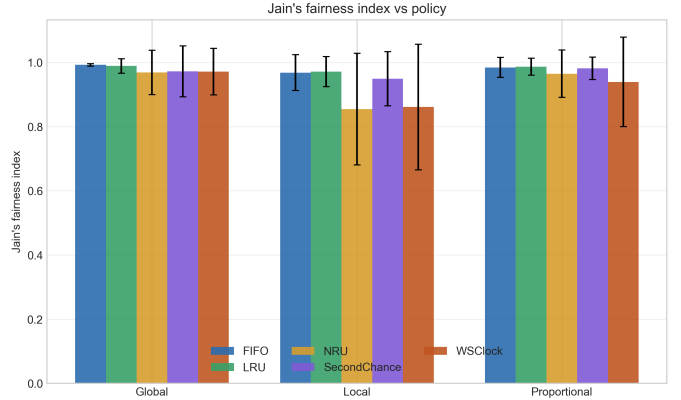


Fig. 4. Jain's fairness index by allocation policy in the final aggregate.

The third threat is the simulator-to-kernel gap. The simulator gives strong control over policies and instrumentation, but it omits many production details such as hardware prefetching, TLB behavior, disk scheduling, NUMA effects, and kernel paging heuristics.

The fourth threat is validation scope. The canonical validation report confirms readable source groups and no duplicate configurations, but it also records non-blocking instrumentation issues in some raw summaries. For that reason, fairness conclusions are stronger directionally than as exact slowdown magnitudes.

IX. FUTURE WORK

Future work will focus on broadening the trace set, improving runtime isolation, and adding an explicit efficiency cost model. A useful next step is a weighted cost function that combines page faults, disk reads, disk writes, replacement activity, and slowdown penalties. This would let future experiments compare whether the policy that minimizes faults also minimizes total execution cost under more realistic storage-cost assumptions.

TABLE IV
FINAL H1/H2/H3 HYPOTHESIS SUMMARY.

Hyp.	Test condition	Baseline → cmp.	Change	Status
H1	High locality, 4 procs, near 1.2x WSS	326.2 → 325.1 faults	-0.34%	Supported
H2	Aggregate-WSS boundary, 4 procs	0.01190 → 0.01886 variance	+58.53%	Supported
H3	Mixed workload, 4 procs, near WSS	2.6449 → 3.2209 slowdown	+21.78%	Not sup.

X. REPRODUCIBILITY AND ARTIFACT AVAILABILITY

The canonical raw campaign is `results/raw/campaign_20260404_140330_266`. The canonical processed outputs are `results/processed/final_aggregate_results.csv`, `results/processed/hypothesis_evaluation.csv`, and `results/processed/validation_report.json`. Report figures live in `results/figures/`, report tables live in `results/tables/`, and the final demo bundle lives in `results/demo/final_bundle`.

The exact rerun commands are recorded in `docs/final_submission_checklist.md`. The workflow is to run the unit tests, inspect the command-line help paths, execute the allocation smoke test, refresh the final demo bundle, rebuild report tables and figures, and run `scripts/build_final_report_artifacts.py`. The Windows quiet wrapper was also verified with output metadata under `tests/\_quiet\_wrapper\_smoke/metadata/`.

XI. CONCLUSION

This report evaluated a custom virtual memory simulator and trace-driven benchmark framework for replacement and allocation policies under memory pressure. The final evidence shows that H1 is supported, H2 is supported, and H3 is not supported. The central implication is that page-fault minimization alone is an incomplete policy-selection criterion: allocation choices can shift fairness and worst-case slowdown even when fault counts improve. The most important next direction is a broader trace set paired with a more explicit efficiency cost model, so future evaluations can compare fault, I/O, runtime, and fairness costs in a single framework.

REFERENCES

- [1] L. A. Belady, “A study of replacement algorithms for a virtual-storage computer,” *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966.
- [2] P. J. Denning, “The working set model for program behavior,” *Communications of the ACM*, vol. 11, no. 5, pp. 323–333, 1968.
- [3] —, “Virtual memory,” *ACM Computing Surveys*, vol. 2, no. 3, pp. 153–189, Sep. 1970.
- [4] A. Basu, J. Gandhi, J. Chang, M. D. Hill, and M. M. Swift, “Efficient virtual memory for big memory servers,” in *Proceedings of the 40th Annual International Symposium on Computer Architecture (ISCA)*, 2013.
- [5] C. H. Park, I. Vougioukas, A. Sandberg, and D. Black-Schaffer, “Page tables: Keeping them flat and hot (cached),” arXiv preprint arXiv:2012.05079, 2020.
- [6] B. Powers, D. Tench, E. D. Berger, and A. McGregor, “Mesh: Compacting memory management for c/c++ applications,” in *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2019, pp. 333–346.

- [7] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, “Producing wrong data without doing anything obviously wrong!” in *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, 2009, pp. 265–276.
- [8] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. John Wiley & Sons, 2019.
- [9] S. Ghandeharizadeh, S. Irani, and J. Lam, “Cache replacement with memory allocation,” in *Proceedings of the 17th Workshop on Algorithm Engineering and Experiments (ALENEX)*, 2015, pp. 1–9.
- [10] D. Schwalb, T. Berning, M. Faust, M. Dreseler, and H. Plattner, “nvm malloc: Memory allocation for nvram,” in *Proceedings of the 6th International Workshop on Accelerating Data Management Systems Using Modern Processor and Storage Architectures*, 2015, pp. 61–72.